

codehive

Python Classes and Objects

Blueprints and Instances in OOP

1. The Blueprint Concept

In Python, almost **everything is an object**. Each object has properties (attributes) and methods (functions).

A **Class** is a blueprint or a constructor and an **Object** is the actual building created from that blueprint.

Why use Classes?

Classes allow you to group related data and functions together, making your code more modular and easier to understand.

2. Defining a Class

To create a class, you use the `class` keyword. Conventionally, class names in Python use **PascalCase** (each word capitalized).

```
# Defining a simple class
class MyClass:
    x = 5 # This is a class property
```

At this stage, `MyClass` is just a definition. It doesn't actually exist in memory as a piece of data until we create an object from it.

3. Creating an Object

Creating an object from a class is called **instantiation**. You call the class name followed by parentheses.

```
# Create an object named p1
p1 = MyClass()

# Access the property 'x'
print(p1.x) # Output: 5
```

Once p1 is created, it has access to all variables defined inside MyClass.

4. Working with Multiple Objects

One of the main strengths of OOP is that you can create an unlimited number of objects from a single class blueprint.

```
p1 = MyClass()  
p2 = MyClass()  
p3 = MyClass()  
  
print(p1.x, p2.x, p3.x)
```

Independence: Each object is a separate instance. Changing a property in p1 (if it's an instance variable) won't necessarily change p2.

5. Deleting Objects

If you need to free up memory or remove an object from your program, you can use the `del` keyword.

Syntax:

```
>> del p1
```

If you attempt to access `p1.x` after the `del` command, Python will raise a **NameError** because the object no longer exists.

6. The pass Statement

In Python, class definitions cannot be logically empty. However, if you are planning a class for the future and haven't written the content yet, use **pass**.

```
class Person:  
    pass # Prevents error in empty class
```

Without pass, Python would raise an `IndentationError`.

7. Properties and Values

Objects are useful because they can store state. You can modify properties of an object after creation.

```
class Car:
    wheels = 4

my_car = Car()
my_car.color = "Red" # Dynamic property assignment

print(my_car.wheels) # 4
print(my_car.color)  # Red
```


8. Object-Oriented Philosophy

By using Classes and Objects, you move away from Procedural Programming (lines of code acting on variables) to Structural Programming.

- **Encapsulation:** Keeping properties and logic together.
- **Reusability:** One class can serve 1,000 objects across different apps.

Codehive

9. Practical Exercise

Knowledge Check

Question: What is the correct syntax for deleting an object named person in Python?

☐ person.delete()

☐ delete person

☒ del person

10. Summary Table

Term	Definition
Class	The Blueprint (Static definition)
Object	The Instance (Active data)
del	Keyword to remove an object
pass	Keyword to allow empty definitions

codehive

Python Architecture Series